

Algoritmia: Dividir i Vèncer

FME-UPC

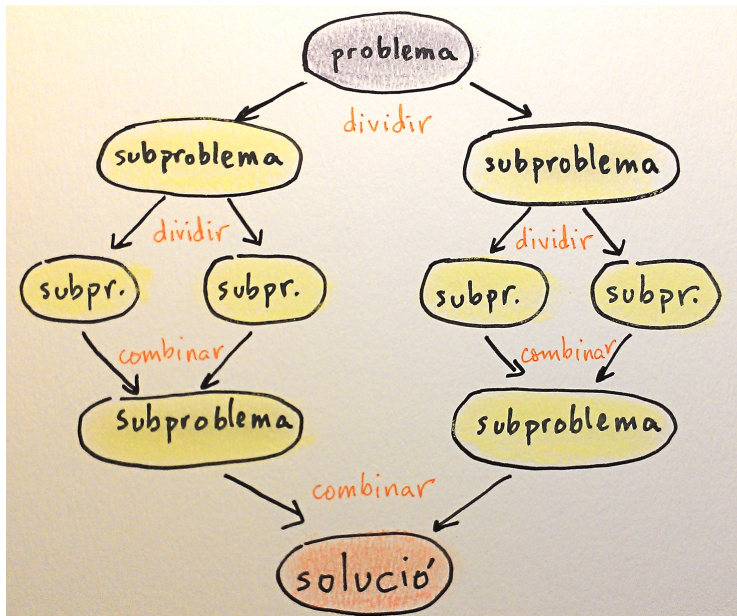
Q1 2024 – 25

Mètode

L'estratègia de **dividir i vèncer** resol un problema en tres passos:

- ① dividint-lo en *subproblemes*, casos més “petits” del mateix problema,
- ② resolent els subproblemes recursivament i
- ③ combinant les respostes de manera adequada.

Gàficament, el mètode es pot representar com mostra la figura.



Els algorismes de *dividir i vèncer* segueixen sovint una mateixa estratègia: ataquen un problema de mida n

- dividint-lo en a subproblemes de mida n/b ,
- resolent els subproblemes recursivament, i
- combinant les respostes,

on $a \geq 1, b > 1$, i el cost de dividir en subproblemes i combinar respostes és $\Theta(n^k)$ per a $k \geq 0$.

Llavors el cost de l'algorisme es pot descriure mitjançant la recurrència

$$T(n) = a \cdot T(n/b) + \Theta(n^k)$$

que es pot resoldre aplicant el teorema mestre de recurrències divisores.

Exemple: Cerca Binaria

```
// Retorna posicio de x al vector ordenat v,  
// -1 si no hi es.  
int cerca_binaria(int x, const vector<int>& v, int i, int j)  
    if (i > j) return -1;  
    int m = (i + j)/2;  
    if (x < v[m]) return cerca_binaria(x, v, i, m-1);  
    if (x > v[m]) return cerca_binaria(x, v, m+1, j);  
    return m;
```

El paràmetre de recursió és $n = j - i + 1$ i per tant el cost és $T(n) = T(n/2) + \Theta(1)$.

Pel teorema mestre de recurrències divisores $T(n) = \Theta(\log n)$.

Exemple: Ordenació per fusió

L'**ordenació per fusió** (*mergesort*) és un bon exemple de *dividir i vèncer* que fa servir un nombre de comparacions gairebé òptim.

Mergesort va ser inventat per John von Neumann l'any 1945.

És un algorisme estable en un doble sentit:

- preserva l'ordre entre valors iguals i
- es comporta igual independentment de com d'ordenada estigui l'entrada.

Exemple: Ordenació per fusió

L'operació clau consisteix en combinar (**fusionar**) dos vectors ordenats en un. Donat un vector T de talla ≥ 2 , l'esquema de l'algorisme és:

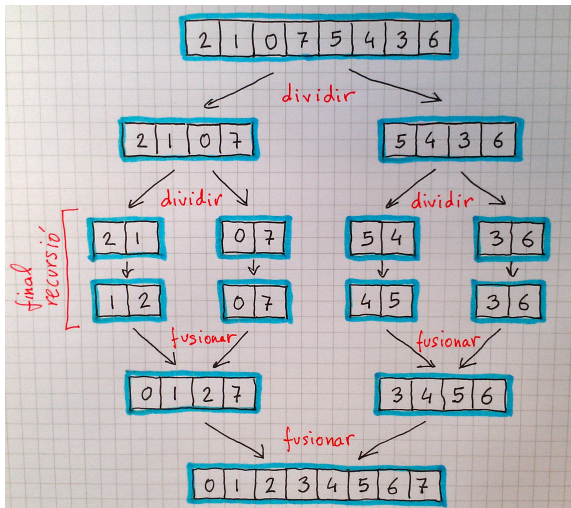
- 1 Partir el vector en dues meitats.
- 2 Ordenar recursivament la primera meitat de T .
- 3 Ordenar recursivament la segona meitat de T .
- 4 Retornar la fusió de les dues meitats.

Exemple: Ordenació per fusió

```
// Ordenacio per fusio
template <typename elem>
void mergesort (vector<elem>& T) {
    mergesort(T, 0, T.size() - 1);
}
```

```
template <typename elem>
void mergesort (vector<elem>& T, int e, int d) {
    if (e < d) {
        int m = (e + d) / 2;
        mergesort(T, e, m);
        mergesort(T, m + 1, d);
        merge(T, e, m, d);
    }
}
```

Exemple: Ordenació per fusió



Aquí, la recursió acaba per a talla 2, a l'algorisme és per a 1. El cor de l'algorisme és fer la fusió de dos vectors ordenats.

Exemple: Ordenació per fusió

```
// Fusio
template <typename elem>
void merge (vector<elem>& T, int e, int m, int d) {
    vector<elem> B(d - e + 1);
    int i = e, j = m + 1, k = 0;
    while (i <= m and j <= d) {
        if (T[i] <= T[j]) B[k++] = T[i++];
        else B[k++] = T[j++];
    }
    while (i <= m) B[k++] = T[i++];
    while (j <= d) B[k++] = T[j++];
    for (k = 0; k <= d-e; ++k) T[e + k] = B[k];
}
```

Donat que el procediment merge és lineal, el cost de l'ordenació per fusió es pot expressar amb la recurrència

$$T(n) = 2T(n/2) + \Theta(n) \quad \text{per a } n > 1$$

i, aplicant el teorema mestre de recurrències divisores, tenim que:

$$T(n) = \Theta(n \log n).$$

Exemple: Ordenació ràpida

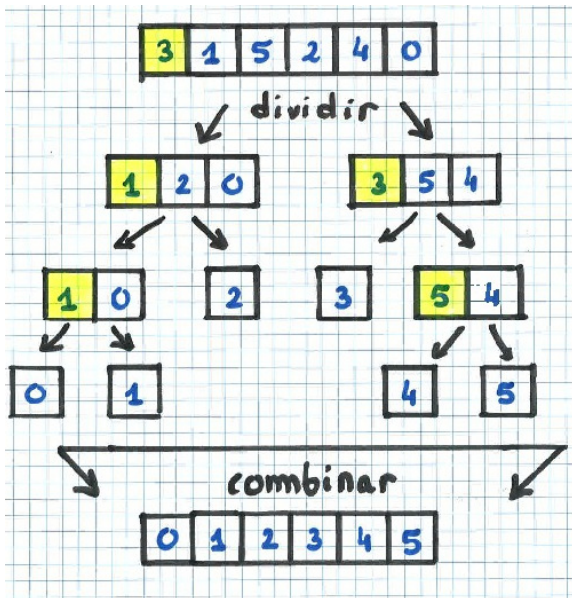
L'**ordenació ràpida**, o *QuickSort*, és l'algorisme d'ordenació genèric més ràpid, com el seu nom indica. Tot i que el seu cost en el cas pitjor és $\Theta(n^2)$, el cas mitjà és $\Theta(n \log n)$, i l'eficiència del seu bucle intern fa que sigui el millor algorisme a la pràctica.

QuickSort va ser inventat per C. A. R. Hoare l'any 1960. Ha estat molt estudiat des de llavors.

Donat un vector T d'almenys 2 elements, l'algorisme bàsic fa el següent:

- 1 Triar un element x de T .
- 2 Dividir T en dos grups disjunts T_1 i T_2 de forma que:
 - T_1 conté elements $\leq x$ de T , i
 - T_2 conté elements $\geq x$ de T .
- 3 Ordenar T_1 i T_2 recursivament.
- 4 Retornar T_1 seguit de T_2 .

Exemple: Ordenació ràpida



Exemple: Ordenació ràpida

```
// Ordenacio rapida
template <typename elem>
void quicksort(vector<elem>& T, int e, int d) {
    if (e < d) {
        int q = partition(T, e, d);
        quicksort(T, e, q);
        quicksort(T, q + 1, d);
    }
}
```

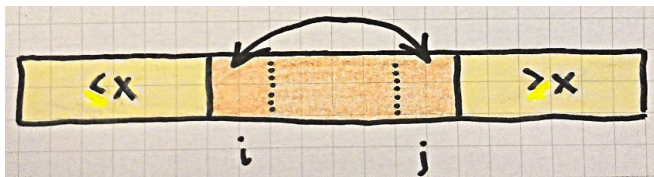
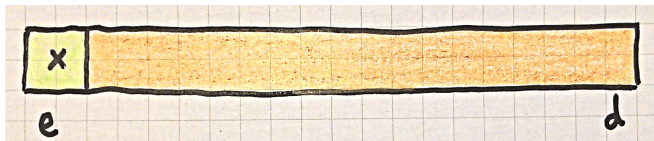
```
template <typename elem>
void quicksort(vector<elem>& T) {
    quicksort(T, 0, T.size() - 1);
}
```

Exemple: Ordenació ràpida

La partició original de Hoare amb el primer element com a pivot és com segueix:

```
//Particio de Hoare
template <typename elem>
int partition (vector<elem>& T, int e, int d) {
    elem x = T[e];
    int i = e - 1;
    int j = d + 1;
    for (;;) {
        while (x < T[--j]);
        while (T[++i] < x);
        if (i >= j) return j;
        swap(T[i], T[j]);
    }
}
```

Exemple: Ordenació ràpida



Exemple: Ordenació ràpida

Sigui $T(n)$ el cost de l'algorisme d'ordenació ràpida amb n elements. Llavors, per a $n \geq 2$,

$$T(n) = T(i) + T(n - i) + \Theta(n),$$

on i és el nombre d'elements de la primera part, i $n - i$ de la segona.

Exemple: Ordenació ràpida

En el **cas pitjor**, el pivot és sempre l'element més petit o el més gran. Com a resultat, una crida recursiva té cost constant i l'altra $T(n - 1)$.

$$T(n) = T(n - 1) + \Theta(n)$$

Resolent la recurrència (directament, o amb el teorema mestre), s'obté:

$$T(n) = \Theta(n^2).$$

En el **cas millor**, el pivot és la mediana del vector i, per tant, els dos subvectors tenen la mateixa mida.

$$T(n) = 2T(n/2) + \Theta(n)$$

Pel teorema mestre de recurrències divisores, tenim que:

$$T(n) = \Theta(n \log n).$$

Exemple: Ordenació ràpida

Per al cas mitjà, suposarem que la partició és aleatòria. Hi ha una probabilitat $\frac{1}{n}$ de tenir una part de mida $i - 1$ per a $1 \leq i \leq n$.

Llavors,

$$T(n) = n + 1 + \frac{2}{n} \left[\sum_{j=0}^{n-1} T(j) \right] + cn \quad (1)$$

Multipliquem l'equació 1 per n :

$$nT(n) = n(n + 1) + 2 \left[\sum_{j=0}^{n-1} T(j) \right]. \quad (2)$$

Per eliminar el sumatori, escrivim el cas $n - 1$

$$(n - 1)T(n - 1) = (n - 1)n + 2 \left[\sum_{j=0}^{n-2} T(j) \right] \quad (3)$$

i restem l'equació (3) de la (2)

$$nT(n) - (n - 1)T(n - 1) = 2n + 2T(n - 1). \quad (4)$$

Exemple: Ordenació ràpida

Reordenant els termes,

$$nT(n) = 2n + (n+1)T(n-1). \quad (5)$$

Dividim l'equació (5) per $n(n+1)$:

$$\frac{T(n)}{n+1} = \frac{2}{n+1} + \frac{T(n-1)}{n}. \quad (6)$$

Substituïm n per tots els valors de $n-1$ fins a 1:

$$\frac{T(n)}{n} = \frac{2}{n+1} + \frac{2}{n} + \frac{T(n-2)}{n-1}. \quad (7)$$

$\vdots$

$$\frac{T(n)}{n} = \frac{2}{n+1} + \frac{2}{n} + \dots + \frac{T(1)}{1}. \quad (8)$$

La suma de totes les equacions (7)–(8) dona

$$\frac{T(n)}{n} = \frac{T(1)}{1} + 2 \sum_{i=1}^{n+1} \frac{1}{i}. \quad (9)$$

Exemple: Ordenació ràpida

D'altra banda, se sap que

$$\sum_{i=2}^n \frac{1}{i} = \ln(n) + \gamma - 1,$$

on $\gamma \approx 0.577$ és la constant d'Euler. Substituint aquest valor en l'equació (9) obtinguda abans

$$\frac{T(n)}{n} = \frac{T(1)}{1} + 2 \sum_{i=2}^n \frac{1}{i}$$

es dedueix que

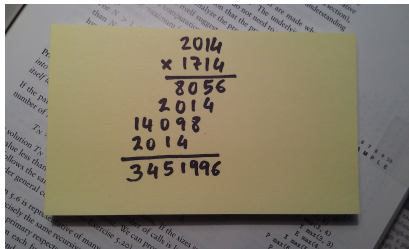
$$\frac{T(n)}{n} = \Theta(\log n)$$

i, per tant,

$$T(n) = \Theta(n \log n).$$

Exemple: Algorisme de Karatsuba

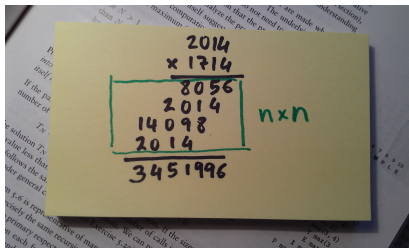
Quin cost té l'algorisme escolar de multiplicació?



Handwritten multiplication of 2014 by 1714 using the standard school algorithm:

$$\begin{array}{r} 2014 \\ \times 1714 \\ \hline 8056 \\ 2014 \\ 14098 \\ 2014 \\ \hline 3451996 \end{array}$$

La resposta és: $\Theta(n^2)$, si n és el nombre de dígit.



Handwritten multiplication of 2014 by 1714, with a box highlighting the intermediate steps and the label $n \times n$:

$$\begin{array}{r} 2014 \\ \times 1714 \\ \hline 8056 \\ 2014 \\ 14098 \\ 2014 \\ \hline 3451996 \end{array} \quad n \times n$$

Exemple: Algorisme de Karatsuba

El matemàtic rus A. Kolmogorov va conjecturar que l'algorisme escolar de multiplicació és òptim.

Conjectura (Kolmogorov, 1952)

Qualsevol algorisme per multiplicar dos nombres de n dígit té cost $\Omega(n^2)$.

Un estudiant de 23 anys de Kolmogorov, Anatolii Alexeevitch Karatsuba, va trobar un algorisme de cost $\Theta(n^{1.585})$.

Refutació (Karatsuba, 1960)

Hi ha un algorisme que multiplica dos nombres de n dígit en temps $\Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$.

Exemple: Algorisme de Karatsuba

L'origen de la idea de l'algorisme de Karatsuba es remunta al s. XVIII. El matemàtic Carl Friedrich Gauss (1777-1855) va observar que malgrat que el producte de dos complexos

$$(a + bi)(c + di) = ac - bd + (bc + ad)i$$

sembla requerir 4 productes de reals, es pot obtenir amb 3. La raó és que

$$bc + ad = (a + b)(c + d) - ac - bd.$$

Avancem ara al s. XX. Suposem que x i y són dos naturals de n bits, amb n potència de dos. Expressem x , y en dues parts:

$$x = \boxed{x_E} \boxed{x_D} = 2^{n/2}x_E + x_D$$

$$y = \boxed{y_E} \boxed{y_D} = 2^{n/2}y_E + y_D$$

Exemple: Algorisme de Karatsuba

Si $x = 10010111_2$ i $y = 11001010_2$ (el subíndex 2 vol dir "en binari"), llavors

$$x = \boxed{x_E} \boxed{x_D} = \boxed{1001_2} \boxed{0111_2}$$

$$y = \boxed{y_E} \boxed{y_D} = \boxed{1100_2} \boxed{1010_2}$$

Ara, el producte

$$xy = (2^{n/2}x_E + x_D)(2^{n/2}y_E + y_D)$$

es pot reescriure com

$$xy = 2^n x_E y_E + 2^{n/2}(x_E y_D + x_D y_E) + x_D y_D$$

Exemple: Algorisme de Karatsuba

Pel teorema mestre de recurrències divisores, sabem que $T(n) = \Theta(n^2)$. Però si apliquem el truc de Gauss, podem obtenir el producte xy fent servir 3 subproductes en lloc de 4 i, així, reduir el cost quadràtic.

Com abans, observem que

$$x_E y_D + x_D y_E = (x_E + x_D)(y_E + y_D) - x_E y_E - x_D y_D.$$

Si ara anomenem

$$a = x_E y_E, \quad b = x_D y_D, \quad c = (x_E + x_D)(y_E + y_D),$$

llavors l'equació

$$xy = 2^n x_E y_E + 2^{n/2} (x_E y_D + x_D y_E) + x_D y_D$$

es pot reescriure com

$$2^n a + 2^{n/2} (c - a - b) + b$$

que només depèn de 3 subproductes.

Exemple: Algorisme de Karatsuba

La nova expressió dóna lloc a un algorisme de cost

$$T(n) = 3T(n/2) + \Theta(n)$$

i, pel teorema mestre de recurrències divisores, sabem que

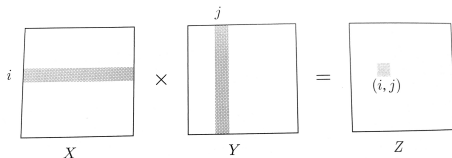
$$T(n) \in \Theta(n^{\log_2 3}) \approx \Theta(n^{1.585}).$$

Exemple: Algorisme de Strassen

El producte de dues matrius X i Y de mida $n \times n$ és una matriu Z de mida $n \times n$ tal que

$$Z_{ij} = \sum_{k=1}^n X_{ik} Y_{kj}.$$

És a dir, Z_{ij} és el producte de la fila i -èsima de X per la columna j -èsima de Y



La fórmula

$$Z_{ij} = \sum_{k=1}^n X_{ik} Y_{kj}.$$

calculada per a cada i, j amb valors $i, j = 1 \dots n$ implica un algorisme $\Theta(n^3)$.

Exemple: Algorisme de Strassen

Es va pensar que el cost $\Theta(n^3)$ era òptim fins al 1969, quan Volker Strassen va anunciar un algorisme de cost $\Theta(n^{\log_2 7}) \approx \Theta(n^{2.81})$.

Una primera idea és que el producte de matrius es pot fer *per blocs*. Dividim X i Y en quatre quadrants cadascuna:

$$X = \begin{bmatrix} A & B \\ C & D \end{bmatrix}, \quad Y = \begin{bmatrix} E & F \\ G & H \end{bmatrix}.$$

Aleshores, es pot veure que

$$XY = \begin{bmatrix} A & B \\ C & D \end{bmatrix} \begin{bmatrix} E & F \\ G & H \end{bmatrix} = \begin{bmatrix} AE + BG & AF + BH \\ CE + DG & CF + DH \end{bmatrix}.$$

Exemple: Algorithme de Strassen

Els productes de matrius es poden calcular aleshores recursivament. Per exemple, per fer el producte XY

$$XY = \begin{bmatrix} 4 & 3 & 1 & 6 \\ 1 & 5 & 2 & 7 \\ 2 & 1 & 5 & 9 \\ 3 & 4 & 2 & 6 \end{bmatrix} \begin{bmatrix} 2 & 6 & 9 & 4 \\ 3 & 2 & 4 & 1 \\ 1 & 1 & 8 & 3 \\ 2 & 1 & 3 & 1 \end{bmatrix}$$

definim les vuit matrius 2×2

$$A = \begin{bmatrix} 4 & 3 \\ 1 & 5 \end{bmatrix}, B = \begin{bmatrix} 1 & 6 \\ 2 & 7 \end{bmatrix}, E = \begin{bmatrix} 2 & 6 \\ 3 & 2 \end{bmatrix}, F = \begin{bmatrix} 9 & 4 \\ 4 & 1 \end{bmatrix},$$

$$C = \begin{bmatrix} 2 & 1 \\ 3 & 4 \end{bmatrix}, D = \begin{bmatrix} 5 & 9 \\ 2 & 6 \end{bmatrix}, G = \begin{bmatrix} 1 & 1 \\ 2 & 1 \end{bmatrix}, H = \begin{bmatrix} 8 & 3 \\ 3 & 1 \end{bmatrix}.$$

Exemple: Algorisme de Strassen

Quin cost té el nou algorisme? Recordem que es basa en el producte

$$XY = \begin{bmatrix} A & B \\ C & D \end{bmatrix} \begin{bmatrix} E & F \\ G & H \end{bmatrix} = \begin{bmatrix} AE + BG & AF + BH \\ CE + DG & CF + DH \end{bmatrix}.$$

El cost $T(n)$ és la suma de fer:

- vuit productes de matrius de mida $n/2$: $8T(n/2)$
- quatre sumes de matrius de mida $n/2$: $\Theta(n^2)$

Per tant, tenim la recurrència

$$T(n) = 8T(n/2) + \Theta(n^2)$$

que, pel teorema mestre de recurrències divisores, dóna

$$T(n) \in \Theta(n^{\log_2 8}) = \Theta(n^3).$$

Exemple: Algorisme de Strassen

Però el nombre de productes es pot reduir a 7.

$$XY = \begin{bmatrix} P_5 + P_4 - P_2 + P_6 & P_1 + P_2 \\ P_3 + P_4 & P_1 + P_5 - P_3 - P_7 \end{bmatrix}$$

on

$$P_1 = A(F - H), \quad P_4 = D(G - E)$$

$$P_2 = (A + B)H, \quad P_5 = (A + D)(E + H)$$

$$P_3 = (C + D)E, \quad P_6 = (B - D)(G + H)$$

$$P_7 = (A - C)(E + F)$$

Fent servir la descomposició de Strassen, obtenim un algorisme amb cost

$$T(n) = 7T(n/2) + \Theta(n^2)$$

que, pel teorema mestre de recurrències divisores, dóna

$$T(n) \in \Theta(n^{\log_2 7}) \approx \Theta(n^{2.81}).$$